

Evaluating PINNsFormer for Solving Partial Differential Equations: A Comparison with Standard PINNs on Five Benchmark Problems

Zohreh Sarmeli Saeedi Maria Mahdavi Layen

Department of Mathematics and Computer Science,
Amirkabir University of Technology (Tehran Polytechnic), Tehran, Iran

Supervisor: Dr. Fatemeh Shakeri

Abstract

Physics-informed neural networks (PINNs) have become a widely used approach for solving partial differential equations (PDEs). However, the performance of standard PINNs can degrade on some problems, particularly stiff PDEs whose solutions contain high-frequency components. One reason that has been proposed for this behavior is the way inputs are processed in the standard PINN architecture: the relationship between different time points is not represented directly in the network structure. The PINNsFormer model attempts to introduce this dependence into the learning process by means of a pseudo-temporal sequence and a self-attention mechanism. In this work, we implement PINNsFormer and compare it with a standard PINN on five PDEs with different characteristics: the Burgers, Allen–Cahn, nonlinear Schrödinger, Kuramoto–Sivashinsky, and Korteweg–de Vries equations. The main comparison metric is the relative error of the numerical solution; computational cost is examined alongside it. In our experiments, PINNsFormer achieved a lower error than the baseline on all five problems, with a reduction in relative error ranging from about 54% to more than 99%, depending on the equation. The largest improvement was observed for the Kuramoto–Sivashinsky equation, where the relative error dropped from about 0.30 for the PINN to 0.0001 for PINNsFormer. This accuracy gain came at a substantial cost: PINNsFormer required on average roughly 13 times longer to train than the standard PINN.

Keywords: physics-informed neural networks, partial differential equations, transformer architecture, attention mechanism

1 Introduction

1.1 Problem Statement

Partial differential equations (PDEs) are an important mathematical tool for describing many phenomena in science and engineering; heat transfer, fluid flow, quantum mechanics, and several biological models, for example, can all be expressed through such equations. Analytical solutions are available for only a limited number of simple problems, and in many real-world settings numerical methods must be used. Finite difference, finite element, and spectral methods are well-established approaches for this purpose. Nevertheless, for problems with high dimensionality or complex geometry, the cost of constructing and maintaining a mesh and the cost of the computations themselves increase considerably.

In parallel with advances in scientific machine learning, the use of neural networks to approximate solutions of differential equations has attracted considerable attention. In these methods, instead of first

discretizing the domain into a mesh, the solution is represented by a neural network, and the derivatives required to form the equation can be computed by automatic differentiation. This property makes neural networks attractive for some high-dimensional problems, although these methods also bring their own limitations and numerical challenges.

1.2 Known Limitations of PINNs

The central idea of the PINN approach is that the network is trained not only on initial and boundary data but also on the residual of the differential equation, which is included in the loss function. Training therefore drives the network toward satisfying both the equation and the conditions of the problem.

Despite this, subsequent studies have shown that PINNs based on a standard multilayer perceptron (MLP) perform poorly on some problems. This difficulty is most often observed for stiff equations, high-frequency components, and multiscale behavior [8]. In such situations, the optimization may converge to an inadequate approximation of the solution. For problems with oscillatory behavior or rapid variations, for instance, the network may struggle to represent high-frequency components and consequently drift toward overly smooth approximations. In addition, differences in the scale of the gradients associated with the different terms of the loss function can unbalance the optimization [9, 10]: some terms dominate training while the others are learned more slowly.

Another relevant point concerns how time dependence is represented in the standard architecture. In an ordinary PINN, each pair (x, t) is fed directly to the network and the output $\hat{u}(x, t)$ is computed. As a result, the relationship between a point and its neighboring time points is not represented explicitly in the network structure. This can be a limitation, particularly for problems in which the behavior of the solution over time is important. By modifying exactly this part of the architecture, namely converting each input into a short sequence and applying an attention mechanism, PINNsFormer attempts to bring information about nearby times into the learning process.

1.3 Objectives

The goal of this work is to implement and evaluate the PINNsFormer framework and to compare its performance with a standard PINN on PDEs. To this end, the method is first reproduced from the original paper, and its performance is then examined on five different equations: Burgers, Allen–Cahn, nonlinear Schrödinger, Kuramoto–Sivashinsky, and Korteweg–de Vries. We also assess the magnitude of the error reduction and how it relates to the characteristics of each problem and to the difficulty of training a PINN.

2 Background and Related Work

2.1 Deep Learning for Partial Differential Equations

Over the past decade, two main directions have emerged for applying deep learning in this field. The first is *operator learning*, which, instead of approximating one specific solution, learns a mapping between function spaces, for example from an initial condition or the parameters of a problem to the solution function of the equation. Prominent methods in this direction include DeepONet [4] and the Fourier Neural Operator (FNO) [5]. DeepONet approximates this mapping using two neural networks, one representing the functional input and one representing the evaluation location, whereas FNO enables the modeling of differential operators by learning in Fourier space. The second direction, which forms the basis of this work, is *physics-informed neural networks*. Rather than learning a general mapping,

it directly approximates the solution of one specific problem by imposing the physical constraint of the equation on the network output.

2.2 Physics-Informed Neural Networks: Foundations and Evolution

The idea of using neural networks to solve differential equations goes back to the work of Lagaris et al. [3] in the 1990s, who used a simple network to solve ordinary and partial differential equations with simple boundary conditions. Later, with the emergence of automatic differentiation in modern deep learning frameworks, Raissi et al. [2] reformulated this idea in its present form under the name physics-informed neural networks (PINNs), and the model quickly became well known. In this model, the solution $u(\mathbf{x}, t)$ is represented by a neural network $\hat{u}_\theta$, and the loss function combines the residual of the equation (computed by automatic differentiation at sampled interior points) with the error in the boundary and initial conditions:

$$\mathcal{L}_{\text{PINN}}(\theta) = \underbrace{\lambda_r \frac{1}{N_r} \sum_{i=1}^{N_r} \|\mathcal{D}[\hat{u}_\theta](\mathbf{x}_i, t_i) - f(\mathbf{x}_i, t_i)\|^2}_{\text{equation residual}} + \underbrace{\lambda_b \frac{1}{N_b} \sum_{i=1}^{N_b} \|\mathcal{B}[\hat{u}_\theta](\mathbf{x}_i, t_i) - g(\mathbf{x}_i, t_i)\|^2}_{\text{boundary/initial condition}}. \quad (1)$$

This simple formulation, which requires neither mesh generation nor labeled data, led the approach to spread rapidly across fields ranging from fluid mechanics to biomedicine. Libraries such as DeepXDE [6] have also made implementations widely accessible to researchers.

2.3 Origins of PINN Failures: From Spectral Bias to Optimization Pathologies

A recurring observation is that, for certain classes of problems, a standard PINN converges to an overly smooth approximation rather than the true solution. The following findings help explain this phenomenon.

Spectral bias of neural networks. Rahaman et al. [7] showed that when approximating a function, neural networks learn low-frequency components faster than high-frequency ones. As a result, approximating oscillatory components and components with rapid variation is harder for the network. Because the solutions of some PDEs, especially in the presence of strong convection or sharp boundary layers, contain high-frequency components, spectral bias can reduce the accuracy of PINNs on such solutions and make training more difficult.

Difficulty of optimizing the loss. Krishnapriyan et al. [8], studying convection and reaction–diffusion equations, showed that PINN failures are not necessarily caused by limited network capacity but can stem from the structure of the composite loss function. As the physical complexity of the problem increases, the loss landscape can become strongly non-convex and contain regions with unfavorable geometry; gradient-based optimizers may then become trapped in undesirable critical points, making convergence to the exact solution difficult. Complementary lines of work address these issues through gradient-flow analysis [9], neural tangent kernel analysis [10], self-adaptive loss weighting [11], and causality-respecting training schemes [12].

2.4 Sequence Modeling and Attention for Time-Dependent Problems

In time-dependent problems, the value of the solution at a point in space and time depends on its values at other time points. The Transformer architecture [13], through the *self-attention* mechanism, can model such dependencies directly among the elements of a sequence, without processing the sequence step by step as recurrent networks do. However, the usual input of a PINN is a single point $(\mathbf{x}, t)$, which has no natural sequential structure.

In the PINNsFormer method [1], each point $(\mathbf{x}, t)$ is converted into a short pseudo-sequence by considering several neighboring time points. An architecture based on self-attention then learns the relationships among these time points and incorporates temporal information into the approximation of the solution. Consequently, instead of processing each space–time point independently, the network can take into account the dependence among several consecutive times during learning.

3 PINNsFormer: Architecture and Mechanisms

3.1 General Problem Formulation

This section introduces the notation and mathematical formulation used in the remainder of the paper. All PDEs considered here follow the general form

$$\mathcal{D}[u(x, t)] = f(x, t), \quad \forall (x, t) \in \Omega, \quad (2)$$

$$\mathcal{B}[u(x, t)] = g(x, t), \quad \forall (x, t) \in \partial\Omega, \quad (3)$$

$$u(x, 0) = h(x), \quad \forall x, \quad (4)$$

where $\Omega \subset \mathbb{R}^2$ is the space–time domain of the problem, $\partial\Omega$ is its boundary, $\mathcal{D}$ is the (linear or nonlinear) differential operator of the equation, and $\mathcal{B}$ is the boundary-condition operator. The goal is to find a function $\hat{u}_\theta : \mathbb{R}^2 \rightarrow \mathbb{R}$, represented by a neural network with parameters θ , that approximately satisfies all three relations. As shown in the remainder of this section, the main difference between the standard PINN and PINNsFormer lies in *how* $\hat{u}_\theta$ is represented and in the *structure* of the loss function, not in the problem formulation itself.

To sample this continuous formulation numerically, three sets of points are defined: residual points $\{(x_i, t_i)\}_{i=1}^{N_r} \subset \Omega$ (to enforce Eq. (2)), boundary points $\{(x_i, t_i)\}_{i=1}^{N_b} \subset \partial\Omega$ (for Eq. (3)), and initial points $\{x_i\}_{i=1}^{N_{ic}}$ (for Eq. (4)). These are the collocation points.

3.2 Architecture Overview

Figure 1 shows the path that a point input follows from the beginning to the end of the network: an input point (x, t) passes in turn through four main components, namely the sequence generator, the mixing layer, the encoder–decoder, and the output layer, and is finally converted into an approximation of the solution.

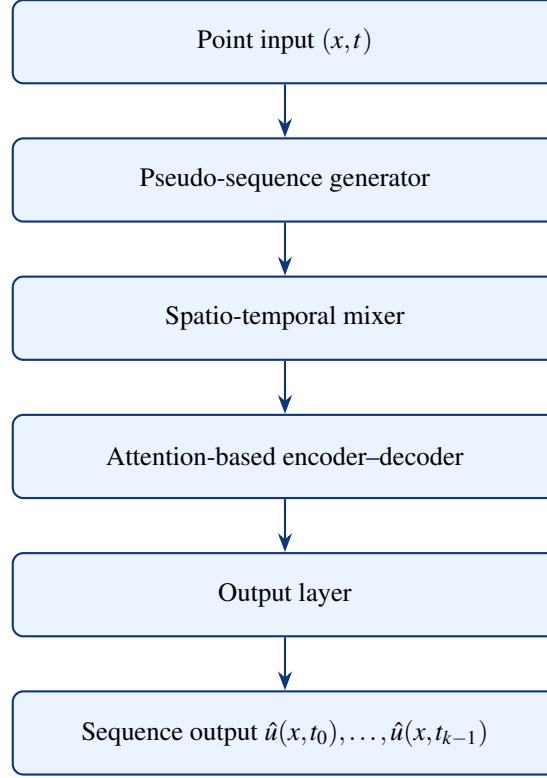


Figure 1: Overall stages of the PINNsFormer architecture.

3.3 Step 1: Pseudo-Sequence Generator

The problem with a standard PINN is that it processes each point (x, t) independently of the other time points, whereas an attention mechanism requires a sequence. Since the natural input of a PDE is a single point, the *pseudo-sequence generator* creates an artificial sequence by extending the input point over several time steps, thereby making the use of attention possible.

Formulation. For a spatial input x and a temporal input t , and with two hyperparameters k (sequence length) and Δt (time step), the following sequence is constructed:

$$[x, t] \xrightarrow{\text{Generator}} \{[x, t_0], [x, t_1], \dots, [x, t_{k-1}]\}, \quad t_j = t + j\Delta t. \quad (5)$$

Note that the spatial value x is held fixed across all steps of the sequence and only the temporal component changes. Figure 2 illustrates this operation for $k = 5$.

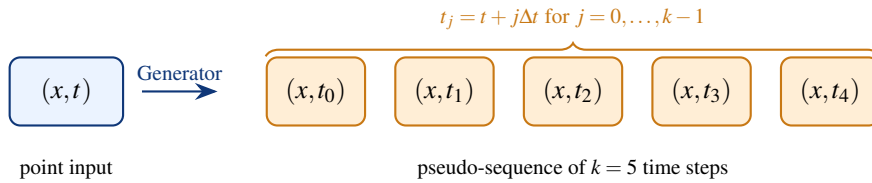


Figure 2: The pseudo-sequence generator. A point input (x, t) is converted into a sequence of length k by repeating the spatial value x and increasing time gradually with a fixed step Δt .

The choice of k and Δt is of considerable practical importance, because these two parameters determine the time horizon ($k\Delta t$) that the model can see directly. As the empirical evidence in Section 4.4.1 suggests, if this horizon is not matched to the actual time scale of the equation's dynamics, it can directly

affect the success or failure of the model.

3.4 Step 2: Spatio-Temporal Mixing

Each element of the sequence is still a two-dimensional vector $[x, t_j]$. This low-dimensional representation is insufficient for the attention mechanism to learn complex relationships; therefore, as in the Transformer architecture, each input is first mapped (embedded) into a higher-dimensional vector space so that the network can better represent and process the more complex features present in the data.

Formulation. This is done with a simple linear layer:

$$\mathbf{h}_j = W_{\text{mix}} [x, t_j] + \mathbf{b}_{\text{mix}}, \quad \mathbf{h}_j \in \mathbb{R}^{d_{\text{model}}}, \quad (6)$$

where W_{mix} and $\mathbf{b}_{\text{mix}}$ are learnable parameters and d_{model} (equal to 32 in this work) is the dimension of the new space. In this way, the sequence of k two-dimensional vectors is converted into a sequence of d_{model} -dimensional vectors, which forms the input to the next stage, the encoder–decoder.

3.5 Step 3: Attention-Based Encoder–Decoder

This component is the core of the architecture; its role is to learn the dependencies among the elements of the sequence. The *self-attention* mechanism is used to determine how strongly each element is related to the other elements of the sequence.

3.5.1 Self-Attention

Suppose we have a sequence of k vectors. In self-attention, the representation of each vector is updated using the information from all elements of the sequence: for each element a query, a key, and a value vector are constructed, and the similarity between the query of one element and the keys of the others determines the weights with which the value vectors are combined. In our implementation, this computation is carried out for a sequence of fixed length $k = 5$, each element of which is mapped to a 32-dimensional vector.

Formulation. For the input sequence matrix $H \in \mathbb{R}^{k \times d_{\text{model}}}$ (each row of which is one of the vectors $\mathbf{h}_j$), the query, key, and value matrices are constructed with separate linear maps:

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V, \quad (7)$$

and the attention output is computed as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \quad (8)$$

where $QK^\top$ computes the similarity matrix between all query–key pairs, the division by $\sqrt{d_k}$ provides numerical stability, and the softmax converts these similarities into weights that sum to 1. Figure 3 gives a schematic view of this computational flow.

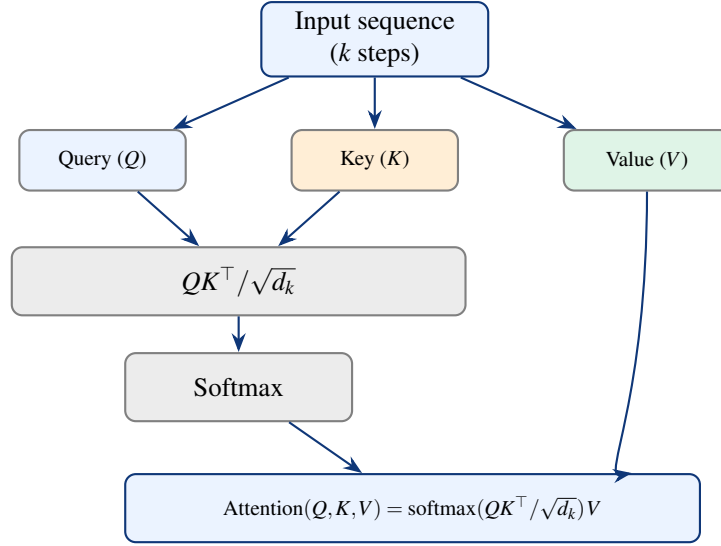


Figure 3: The self-attention mechanism.

In practice, self-attention is executed simultaneously in several independent heads, which is called *multi-head attention*. Each head can learn a different type of dependency among the elements of the sequence, and at the end the outputs of the heads are combined. In this work, 2 attention heads are used.

3.5.2 Encoder–Decoder Structure

In the Transformer architecture, the encoder converts the input sequence into a richer representation and the decoder uses this representation to produce the output. In a PINN, the goal is not to predict the next time step; rather, the network must approximate the value of the solution at the desired space–time points. The decoder of PINNsFormer therefore differs from that of the standard Transformer. In this structure, the decoder uses the input embedding directly and, through an encoder–decoder attention layer, computes the relationship between the input representation and the encoder output. A feed-forward layer then processes this representation and produces the final output.

Formulation. The encoder output with N consecutive layers ($N = 1$ in this work, following the original paper) is computed as

$$Z^{(0)} = H, \quad (9)$$

$$Z^{(l)} = \text{FFN}\left(Z^{(l-1)} + \text{Attention}\left(Z^{(l-1)}, Z^{(l-1)}, Z^{(l-1)}\right)\right), \quad l = 1, \dots, N, \quad (10)$$

where FFN is a two-layer feed-forward network. In the decoder, the query vectors are computed from the decoder’s input representation, while the key and value vectors are computed from the final encoder output $Z^{(N)}$. The output of this attention layer is then passed through an FFN and, after processing, serves as the final decoder representation.

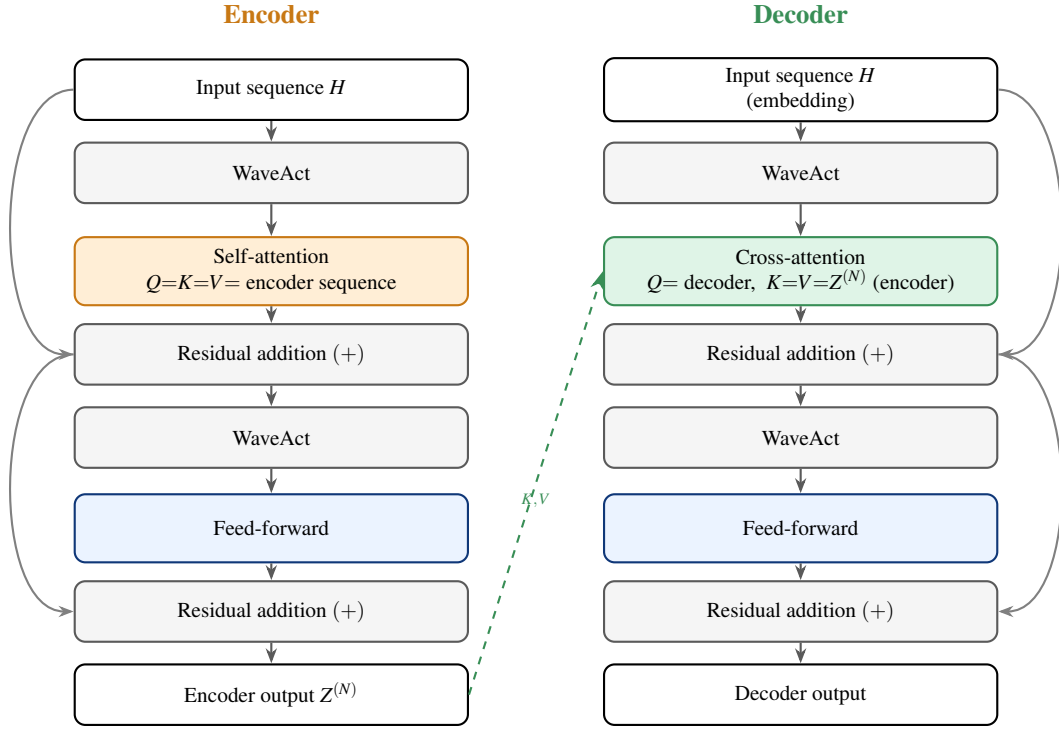


Figure 4: Detailed structure of the encoder and decoder in PINNsFormer.

3.5.3 The Wavelet Activation Function

Activations such as ReLU are unsuitable for PINNs because their derivative is discontinuous at zero, which can cause problems when computing higher-order derivatives of the network. The Tanh activation is smooth in this respect and has higher-order derivatives, but its output is bounded to the interval $[-1, 1]$, which may limit the approximation of oscillatory functions or functions with large amplitude. To overcome this limitation, the authors of the original paper [1] proposed an activation called *Wavelet*, designed with inspiration from the Fourier representation.

Formulation. The activation is defined as

$$\text{Wavelet}(x) = \omega_1 \sin(x) + \omega_2 \cos(x), \quad (11)$$

where ω_1, ω_2 are learnable (not fixed) parameters that allow the network to adjust the amplitude and the sine/cosine mixture to each problem. Figure 5 shows the shape of this function (for the sample values $\omega_1 = 1, \omega_2 = 0.5$) in comparison with Tanh.

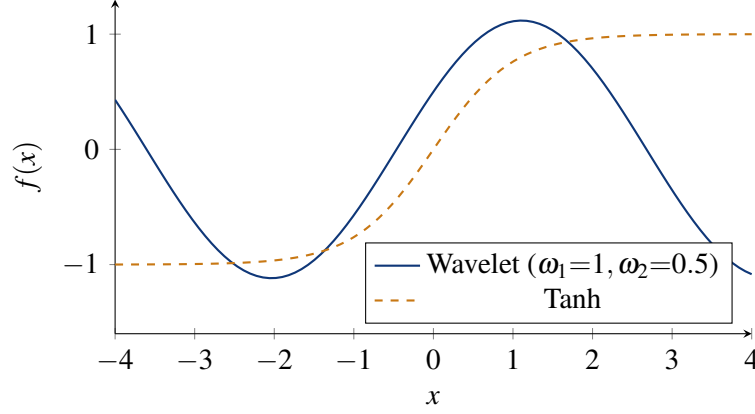


Figure 5: Comparison of the Wavelet activation function with Tanh. Unlike Tanh, which saturates monotonically to the limiting values ± 1 , Wavelet is oscillatory and non-monotonic, which may make it better suited to approximating oscillatory functions.

Theoretical property: universal approximation. This property, namely the ability of a two-layer network of sufficient width with Wavelet activation to approximate any real-valued function to arbitrary accuracy, is proved in the original paper [1] using the real Fourier representation. We cite this result from the paper and do not provide an independent proof in this work. The general idea of the proof rests on the Fourier representation of a function and the approximation of the corresponding integral by a finite sum.

3.6 Step 4: Output Layer

After the sequence has been processed by the encoder and decoder, each element of the sequence is a d_{model} -dimensional vector. The output layer maps this vector to the approximate value of the PDE solution.

Formulation. The output layer is an MLP applied independently to each element of the sequence:

$$\hat{u}(x, t_j) = \text{MLP}(\mathbf{z}_j), \quad j = 0, \dots, k-1, \quad (12)$$

where $\mathbf{z}_j$ is the j -th element of the decoder output. In this way the network produces an approximate value for each of the k times in the sequence. The first value, $\hat{u}(x, t_0) = \hat{u}(x, t)$, is the desired solution at the original input point.

3.7 Number of Parameters and Computational Complexity

Number of parameters. With the settings used in this work (Table 1), the numbers of learnable parameters of the two models are of the same order. PINNsFormer with $k = 5$, $d_{\text{model}} = 32$, and $d_{\text{hidden}} = 512$ has about 454,000 parameters, while the standard PINN with four hidden layers of width 512 has about 527,000. This closeness means that the comparison of the two methods focuses more on differences in architecture than on differences in network capacity.

Computational complexity. In PINNsFormer, each input point is converted into a sequence of k elements, and the number of network evaluations therefore increases relative to the standard PINN. Moreover, the self-attention mechanism for a sequence of length k has a computational cost of $O(k^2 d_{\text{model}})$. Since $k = 5$ is used in this work, this cost remains limited; however, because increasing k raises the cost of self-attention quadratically, a short sequence length is important from a computational standpoint.

3.8 Sequence Loss Function

Since the output of PINNsFormer is produced as a sequence, the PINN loss function must also be defined consistently with this structure. For all elements of the sequence, i.e., the points (x, t_j) , the equation and boundary conditions can be enforced, because these points lie in the domain or on the boundary of the problem. The initial condition, however, is defined only at $t = 0$ and is therefore applied only to the first element of the sequence.

Formulation. Accordingly, the three terms of the loss function are defined as

$$\mathcal{L}_{res} = \frac{1}{kN_{res}} \sum_{i=1}^{N_{res}} \sum_{j=0}^{k-1} \left\| \mathcal{D}[\hat{u}(x_i, t_j)] - f(x_i, t_j) \right\|^2, \quad (13)$$

$$\mathcal{L}_{bc} = \frac{1}{kN_{bc}} \sum_{i=1}^{N_{bc}} \sum_{j=0}^{k-1} \left\| \mathcal{B}[\hat{u}(x_i, t_j)] - g(x_i, t_j) \right\|^2, \quad (14)$$

$$\mathcal{L}_{ic} = \frac{1}{N_{ic}} \sum_{i=1}^{N_{ic}} \left\| \hat{u}(x_i, t_0) - h(x_i) \right\|^2, \quad (15)$$

and the final loss is the weighted sum of these three terms:

$$\mathcal{L}_{PINNsFormer} = \lambda_{res} \mathcal{L}_{res} + \lambda_{ic} \mathcal{L}_{ic} + \lambda_{bc} \mathcal{L}_{bc}. \quad (16)$$

All k steps of the sequence contribute to $\mathcal{L}_{res}$ and $\mathcal{L}_{bc}$, but only the first step, which coincides exactly with $t = 0$, is used in $\mathcal{L}_{ic}$. In all experiments of this work, $\lambda_{res} = \lambda_{ic} = \lambda_{bc} = 1$.

3.9 Training Algorithm

Combining the architecture (Sections 3.3–3.5 and the output layer) with the sequence loss (Section 3.8), Algorithm 1 summarizes the whole training procedure, from the preparation of collocation points to convergence, in pseudocode. This algorithm corresponds exactly to the code that was run for each equation in the experiments below.

Algorithm 1 Training PINNsFormer

- 1: Initialize parameters θ using the Xavier Uniform method
 - 2: Sample collocation points $\{(x_i, t_i)\}_{i=1}^{N_r}$, $\{(x_i, t_i)\}_{i=1}^{N_b}$, $\{x_i\}_{i=1}^{N_{ic}}$
 - 3: Expand each point via the pseudo-sequence generator (Section 3.3) into a k -tuple sequence
 - 4: **while** not converged and iter < iter_{max} **do**
 - 5: $\hat{u}(x_i, t_j) \leftarrow \text{PINNsFormer}_{\theta}(x_i, t_j)$ for all points and $j = 0, \dots, k-1$
 - 6: Compute the required derivatives ($u_t, u_x, u_{xx}, \dots$) via automatic differentiation
 - 7: Compute $\mathcal{L}_{res}, \mathcal{L}_{bc}, \mathcal{L}_{ic}$ according to Eqs. (13)–(15)
 - 8: $\mathcal{L} \leftarrow \lambda_{res} \mathcal{L}_{res} + \lambda_{ic} \mathcal{L}_{ic} + \lambda_{bc} \mathcal{L}_{bc}$
 - 9: $\theta \leftarrow \theta - \text{L-BFGS-step}(\nabla_{\theta} \mathcal{L})$ ▷ with strong Wolfe line search
 - 10: iter $\leftarrow$ iter + 1
 - 11: **end while**
 - 12: **return** θ
-

An important point in line 5 of this algorithm is that the model is evaluated for *all* k steps of the sequence, not just the first, because computing $\mathcal{L}_{res}$ and $\mathcal{L}_{bc}$, unlike $\mathcal{L}_{ic}$, requires all steps. The equivalent algorithm for the standard PINN (Section 3.10) has the same general structure, except that the sequence-expansion step (line 3) is absent and $\mathcal{L}_{res}$ and $\mathcal{L}_{bc}$ are computed pointwise rather than over sequences.

3.10 Comparison with the Standard PINN Baseline

The baseline used in this work is a standard PINN: an MLP with four hidden layers of width 512 and Tanh activation that receives each point (x, t) individually. Table 1 summarizes the architectural differences between the two models at a glance.

Table 1: Structural comparison of the standard PINN and PINNsFormer.

Feature	Standard PINN	PINNsFormer
Input processing unit	single point (x, t)	sequence of length k
Mechanism for temporal dependence	none	self-attention
Activation function	Tanh	Wavelet
Initial/boundary terms in the loss	pointwise	sequence-wise (mean over k steps)
Number of parameters (approx.)	$\sim 527,000$	$\sim 454,000$
Relative computational overhead ($k=5$)	$1 \times$ (baseline)	$\sim 2.9 \times$

3.11 Evaluation Metrics

To quantify the accuracy of each model with respect to the reference solution u , two standard relative-error metrics used for PINNs are employed:

$$\text{rMAE} = \frac{\sum_{n=1}^N |\hat{u}(x_n, t_n) - u(x_n, t_n)|}{\sum_{n=1}^N |u(x_n, t_n)|}, \quad (17)$$

$$\text{rRMSE} = \sqrt{\frac{\sum_{n=1}^N |\hat{u}(x_n, t_n) - u(x_n, t_n)|^2}{\sum_{n=1}^N |u(x_n, t_n)|^2}}, \quad (18)$$

where N is the number of evaluation grid points. Both metrics take values in $[0, \infty)$; the closer the value is to zero, the more accurate the model. In practice, a value close to 1 usually indicates that the model has not converged properly and that its output is only weakly correlated with the true solution, although this threshold is neither exact nor absolute and should be interpreted together with the plotted output.

3.12 Limitations of the PINNsFormer Architecture

Dependence on the hyperparameters k and Δt . In the pseudo-sequence generator, the sequence length k and the time spacing Δt are fixed in advance and determine the model’s time horizon. A fixed choice of these parameters is therefore not necessarily appropriate for problems with different time scales, and tuning them depends on the characteristics of the problem.

Memory cost of higher-order derivatives. Computing higher-order derivatives with autograd requires retaining the computational graph at every differentiation step. In PINNsFormer this cost is larger because of the attention layers and the sequential processing, and it can create memory limitations for problems involving high-order derivatives. In this work, the same issue forced a reduction in the number of collocation points for the equation with a fourth-order derivative.

No explicit bound on the Wavelet output. Unlike Tanh, whose output lies in $[-1, 1]$, the parameters of the Wavelet activation are not explicitly constrained and may take large values during training. This property provides greater flexibility but may increase the likelihood of numerical instability, particularly during optimization.

4 Equations Studied and Results

All experiments use the model settings described in Section 3 ($k = 5$, $d_{\text{model}} = 32$, 2 attention heads, $N = 1$, $d_{\text{hidden}} = 512$) and the baseline described in Section 3.10. The reference solutions for four of the equations are taken from the data files released with the original PINN paper [2]; for the Kuramoto–Sivashinsky equation a closed-form analytical solution is used.

4.1 Burgers’ Equation

The viscous Burgers’ equation [14] is a classical problem among nonlinear differential equations. Because it produces a region of rapid variation and large gradients (a near-shock) close to $x = 0$, particularly for large times, it is a challenging problem for numerical and deep learning methods:

$$\begin{cases} \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} - \frac{0.01}{\pi} \frac{\partial^2 u}{\partial x^2} = 0, & x \in [-1, 1], t \in [0, 1], \\ u(x, 0) = -\sin(\pi x), \\ u(-1, t) = u(1, t) = 0. \end{cases} \quad (19)$$

The settings of this experiment are given in Table 2.

Table 2: Experimental settings for Burgers’ equation.

Collocation point density ($N_x \times N_t$)	51×51
Training iterations	500
Reference solution source	burgers_shock.mat (Raissi et al., 2019)

4.1.1 Results and Analysis

Table 3: Final relative errors and training time for Burgers’ equation.

Model	rMAE	rRMSE	Training time
Standard PINN	0.0262	0.0695	4 min 1 s
PINNsFormer	0.0120	0.0556	28 min 25 s

Final values of the loss components. The standard PINN reached $\mathcal{L}_{\text{res}} = 1.03 \times 10^{-3}$ and PINNsFormer reached $\mathcal{L}_{\text{res}} = 9.07 \times 10^{-5}$; that is, PINNsFormer satisfied the equation residual about one order of magnitude better.

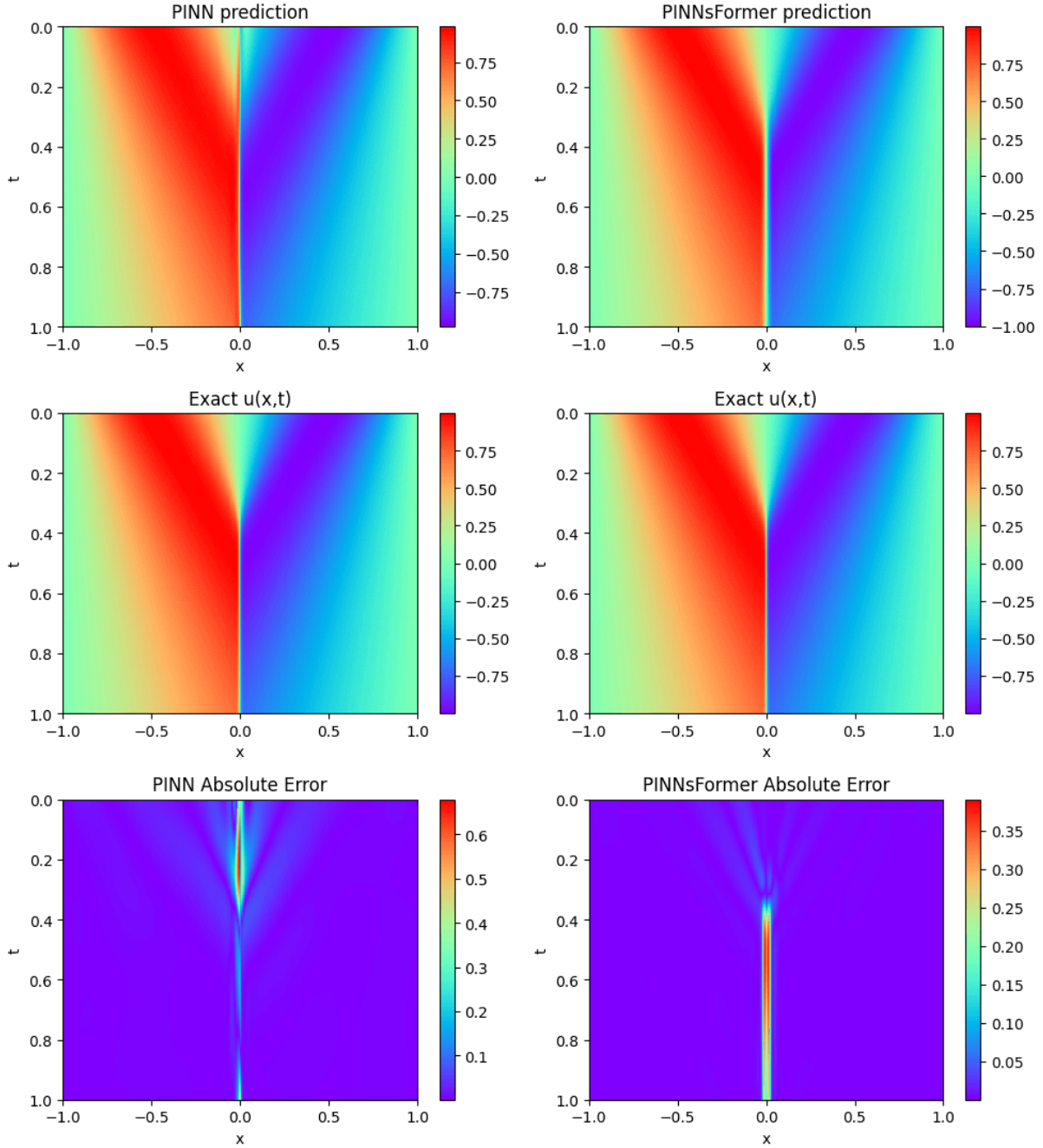


Figure 6: Comparison of the PINN and PINNsFormer predictions with the reference solution, together with the absolute error, for Burgers' equation.

The rMAE decreased from 0.0262 to 0.0120 with PINNsFormer, an improvement of about 54% over the standard PINN. Nevertheless, the standard PINN also performed acceptably on this equation and did not fail seriously; its error (0.0262) lies in a range that is still considered a reasonable approximation of the solution. The improvement observed for Burgers' equation should therefore be regarded as a gradual increase in accuracy rather than the resolution of a failure case.

4.2 Allen–Cahn Equation

The Allen–Cahn equation [15] is one of the challenging problems for PINN methods because of its small diffusion coefficient and a stiff nonlinear reaction term. The equation also has two stable states, $u = \pm 1$,

which can compete during training and drive the network toward local minima.

$$\begin{cases} \frac{\partial u}{\partial t} - 0.0001 \frac{\partial^2 u}{\partial x^2} + 5u^3 - 5u = 0, & x \in [-1, 1], t \in [0, 1], \\ u(x, 0) = x^2 \cos(\pi x), \\ u(-1, t) = u(1, t), \\ u_x(-1, t) = u_x(1, t). \end{cases} \quad (20)$$

Table 4: Experimental settings for the Allen–Cahn equation.

Collocation point density ($N_x \times N_t$)	51×51
Training iterations	500
Reference solution source	AC.mat (Raissi et al., 2019)

4.2.1 Results and Analysis

Table 5: Final relative errors and training time for the Allen–Cahn equation.

Model	rMAE	rRMSE	Training time
Standard PINN	0.9916	0.9910	1 min 28 s
PINNsFormer	0.3790	0.6100	32 min 53 s

Final values of the loss components. The standard PINN reached $\mathcal{L}_{res} = 1.15 \times 10^{-2}$ and $\mathcal{L}_{ic} = 1.36 \times 10^{-1}$; the initial-condition error is roughly one order of magnitude larger than the residual error, which indicates that the model struggles even to fit the initial condition. For the same two terms, PINNsFormer reached $\mathcal{L}_{res} = 1.12 \times 10^{-4}$ and $\mathcal{L}_{ic} = 6.78 \times 10^{-4}$, a reduction of about two orders of magnitude in both, which is consistent with the 62% reduction in rMAE.

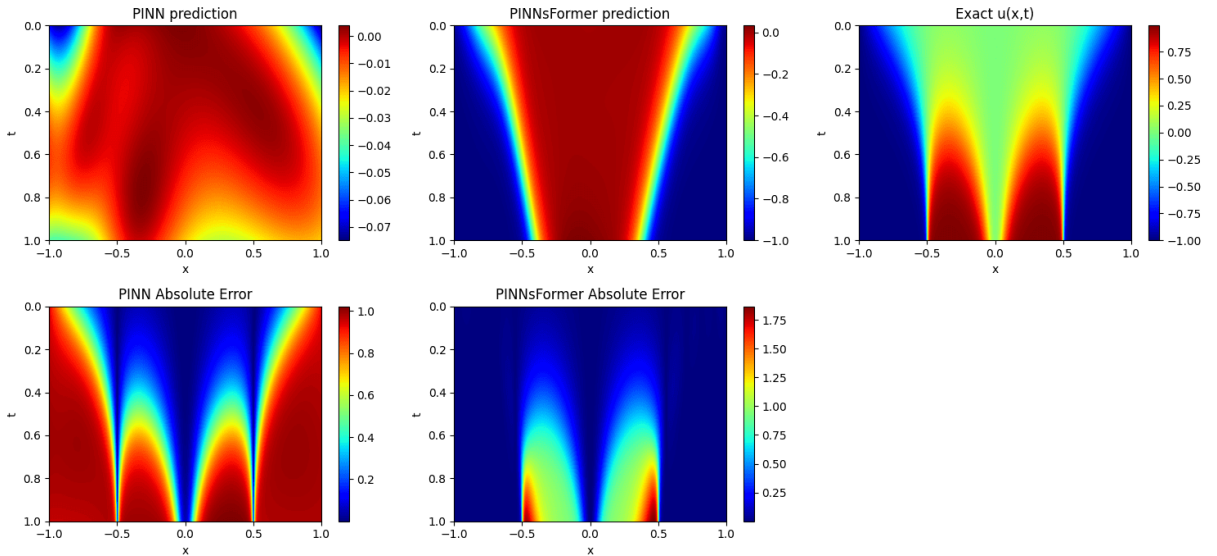


Figure 7: Comparison of the PINN and PINNsFormer predictions with the reference solution for the Allen–Cahn equation. Note that the standard PINN has not learned any meaningful temporal structure, whereas PINNsFormer has reproduced the geometry of the interfaces to a large extent.

Among the five problems studied, this equation posed the greatest challenge to the standard PINN; a value of $\text{rMAE} \approx 0.99$ effectively means that the model failed to approximate the solution. PINNsFormer reduced this value to $\text{rMAE} \approx 0.38$, a reduction of about 62%, and was able to reconstruct a considerable part of the structure of the moving interfaces, although in some regions it erred in identifying the correct stable state between $+1$ and -1 . For this equation, modeling temporal dependencies reduced the error appreciably, but $\text{rMAE} \approx 0.38$ is still a substantial error, and it cannot be claimed that the problem was solved completely.

4.3 Nonlinear Schrödinger Equation

The nonlinear Schrödinger equation [16] is one of the main benchmarks in early PINN studies. Because its solution is complex-valued, it is written as $h = u + iv$, and the network output therefore consists of two independent components, the real part and the imaginary part of the solution:

$$\begin{cases} ih_t + 0.5h_{xx} + |h|^2h = 0, & x \in [-5, 5], t \in [0, \pi/2], \\ h(x, 0) = 2 \operatorname{sech}(x), \\ h(-5, t) = h(5, t), \\ h_x(-5, t) = h_x(5, t). \end{cases} \quad (21)$$

Separating the real and imaginary parts yields the following two real residual equations:

$$f_u \equiv u_t + 0.5v_{xx} + (u^2 + v^2)v = 0, \quad f_v \equiv v_t - 0.5u_{xx} - (u^2 + v^2)u = 0.$$

Table 6: Experimental settings for the Schrödinger equation.

Collocation point density ($N_x \times N_t$)	51×51
Training iterations	500
Reference solution source	NLS.mat (Raissi et al., 2019)
Evaluation quantity	modulus of the complex solution, $ h = \sqrt{u^2 + v^2}$

4.3.1 Results and Analysis

Table 7: Final relative errors and training time for the Schrödinger equation.

Model	rMAE	rRMSE	Training time
Standard PINN	0.0601	0.0926	7 min 12 s
PINNsFormer	0.0032	0.0047	49 min 59 s

Final values of the loss components. Unlike for most of the other equations, here the equation residual of the standard PINN ($\mathcal{L}_{res} = 1.07 \times 10^{-5}$) was slightly lower than the corresponding value for PINNsFormer ($\mathcal{L}_{res} = 1.14 \times 10^{-5}$). The considerable improvement in rMAE for this equation therefore cannot be attributed to a lower equation residual; a more important part of it stems from the reduction of the initial-condition error (from $\mathcal{L}_{ic} = 5.03 \times 10^{-6}$ to 1.77×10^{-6}) and from a more uniform distribution of the error across the domain.

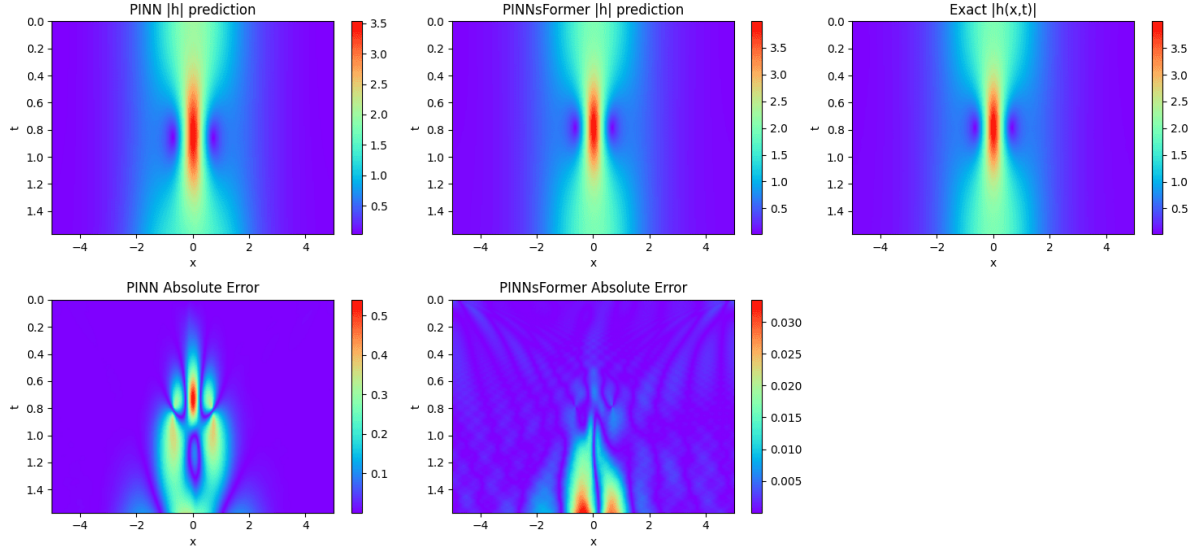


Figure 8: Comparison of the PINN and PINNsFormer predictions with the reference solution over the domain of $|h(x,t)|$ for the Schrödinger equation.

The Schrödinger equation gave the best quantitative result of the study for PINNsFormer: rMAE decreased from 0.0601 to 0.0032, i.e., by about 95%. Both models were able to reconstruct the overall shape of the soliton, but PINNsFormer was more accurate in regions of rapid amplitude variation, further evidence that the self-attention mechanism is effective for nonlinear, continuous temporal dynamics.

4.4 Kuramoto–Sivashinsky Equation

The Kuramoto–Sivashinsky equation [17] is a **fourth-order** PDE used to describe chaotic phenomena and flame-front instabilities. In its general form, the equation contains derivatives up to fourth order, and setting the coefficient of the third-order derivative to zero reduces it to the standard Kuramoto–Sivashinsky form. The general form of the equation is

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + \alpha \frac{\partial^2 u}{\partial x^2} + \beta \frac{\partial^3 u}{\partial x^3} + \gamma \frac{\partial^4 u}{\partial x^4} = 0, \quad x \in [-10, 10], t \in [0, 1], \quad (22)$$

with the initial condition $u(x, 0) = h(x)$ and Dirichlet and Neumann boundary conditions

$$u(a, t) = f_1(t), \quad u(b, t) = f_2(t), \quad u_x(a, t) = g_1(t), \quad u_x(b, t) = g_2(t),$$

where f_1, f_2, g_1, g_2 are taken directly from the value and derivative of the exact analytical solution at the boundaries (not a periodic condition). With $\alpha = 1$, $\beta = 4$, $\gamma = 1$, the exact solution used is an **analytical moving pulse**:

$$u(x, t) = 11 + 15 \tanh(\theta) - 15 \tanh^2(\theta) - 15 \tanh^3(\theta), \quad \theta = -\frac{1}{2}x + t.$$

The validity of this solution was verified by symbolic computation prior to implementation (the equation residual is exactly zero).

Table 8: Experimental settings for the Kuramoto–Sivashinsky equation.

Collocation point density ($N_x \times N_t$)	35×35 *
Training iterations	500
Reference solution source	closed-form analytical solution

* Because of GPU memory limits caused by the depth of the computational graph of the fourth-order derivative, this density was reduced relative to the other equations (51×51).

4.4.1 Results and Analysis

Table 9: Final relative errors and training time for the Kuramoto–Sivashinsky equation.

Model	rMAE	rRMSE	Training time
Standard PINN	0.3012	0.5124	2 min 53 s
PINNsFormer	0.0001	0.0001	53 min 57 s

Final values of the loss components. The standard PINN reached $\mathcal{L}_{res} = 4.02 \times 10^{-2}$, $\mathcal{L}_{bc} = 5.53 \times 10^{-3}$, and $\mathcal{L}_{ic} = 3.14 \times 10^{-2}$; PINNsFormer attained far smaller values in all three terms ($\mathcal{L}_{res} = 1.35 \times 10^{-5}$, $\mathcal{L}_{bc} = 6.16 \times 10^{-7}$, $\mathcal{L}_{ic} = 4.21 \times 10^{-7}$).

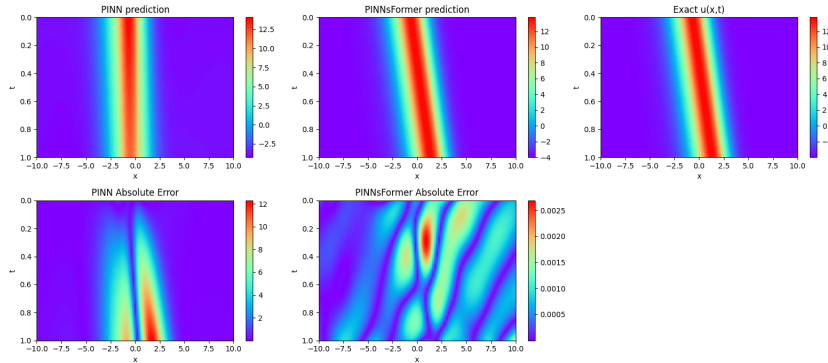


Figure 9: Comparison of the PINN and PINNsFormer predictions with the reference solution for the Kuramoto–Sivashinsky equation. The standard PINN fails to follow the motion of the pulse and effectively keeps it fixed near its initial position; PINNsFormer reproduces the pulse motion with a very small absolute error (at most about 0.0025).

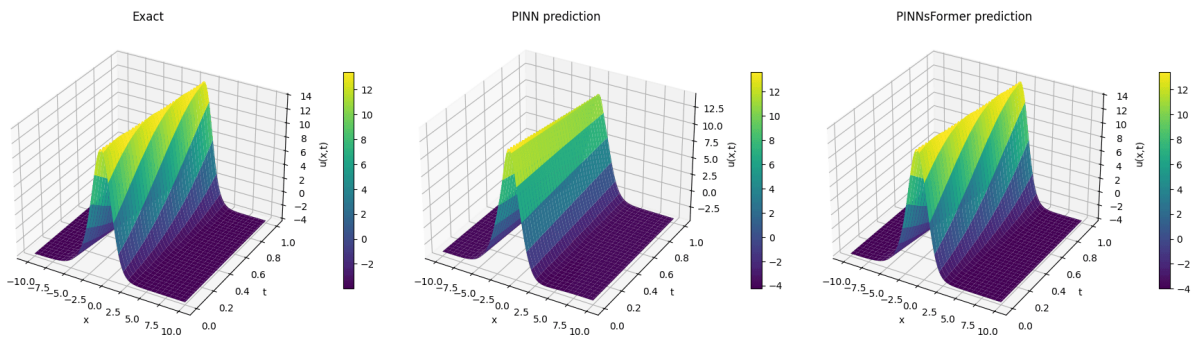


Figure 10: Three-dimensional view of $u(x,t)$ for the exact solution, the PINN prediction, and the PINNsFormer prediction for the Kuramoto–Sivashinsky equation.

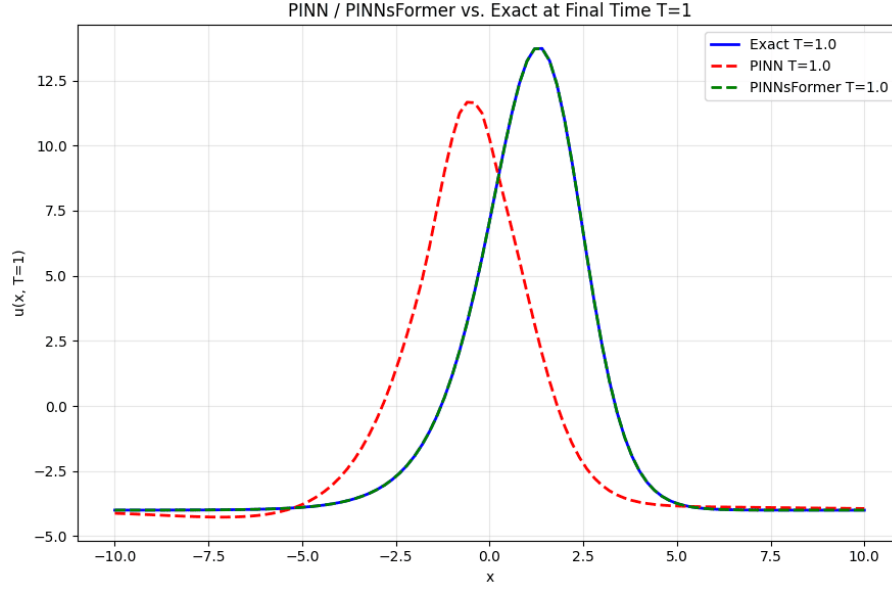


Figure 11: Comparison of the exact solution, PINN, and PINNsFormer at the final time $T = 1$. The PINNsFormer curve effectively coincides with the exact solution, whereas the PINN does not reproduce the position and amplitude of the pulse correctly.

For the Kuramoto–Sivashinsky equation, PINNsFormer reduced the rMAE by more than 99.9% (0.3012 versus 0.0001), the largest improvement among the five equations. At the final time $T = 1$, the standard PINN did not even estimate the position of the pulse correctly: its predicted peak was near $x \approx -0.5$, whereas the true pulse was located at $x \approx 1.3$. The PINNsFormer solution, in contrast, matched the exact solution almost perfectly.

This improvement may be related to the choice of the sequence time horizon, $k\Delta t = 5 \times 10^{-4}$, which could be consistent with the time scale of the pulse motion in this problem and may allow the self-attention mechanism to learn the temporal dependencies that matter in the dynamics of the equation. In other words, the strong performance of PINNsFormer on this equation depends, at least in part, on this compatibility between the physical scale of the problem and the fixed sequence hyperparameters. We note that this interpretation is a hypothesis: no ablation over k and Δt was carried out to test it.

4.5 Korteweg–de Vries Equation

The KdV equation [18], another core problem of the foundational PINN paper, contains a third-order derivative:

$$\begin{cases} \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + 0.0025 \frac{\partial^3 u}{\partial x^3} = 0, & x \in [-1, 1], t \in [0, 1], \\ u(x, 0) = \cos(\pi x), \\ u(-1, t) = u(1, t), \\ u_x(-1, t) = u_x(1, t). \end{cases} \quad (23)$$

Table 10: Experimental settings for the KdV equation.

Collocation point density ($N_x \times N_t$)	51×51
Training iterations	500
Reference solution source	KdV.mat (Raissi et al., 2019)

4.5.1 Results and Analysis

Table 11: Final relative errors and training time for the KdV equation.

Model	rMAE	rRMSE	Training time
Standard PINN	0.0820	0.0857	10 min 38 s
PINNsFormer	0.0184	0.0181	1 h 49 min

Final values of the loss components. The standard PINN reached $\mathcal{L}_{res} = 1.55 \times 10^{-3}$ and PINNsFormer reached $\mathcal{L}_{res} = 8.83 \times 10^{-5}$, a reduction of roughly one order of magnitude, accompanied by similar improvements in the boundary-condition term (3.71×10^{-5} to 4.50×10^{-6}) and the initial-condition term (1.01×10^{-3} to 4.66×10^{-5}).

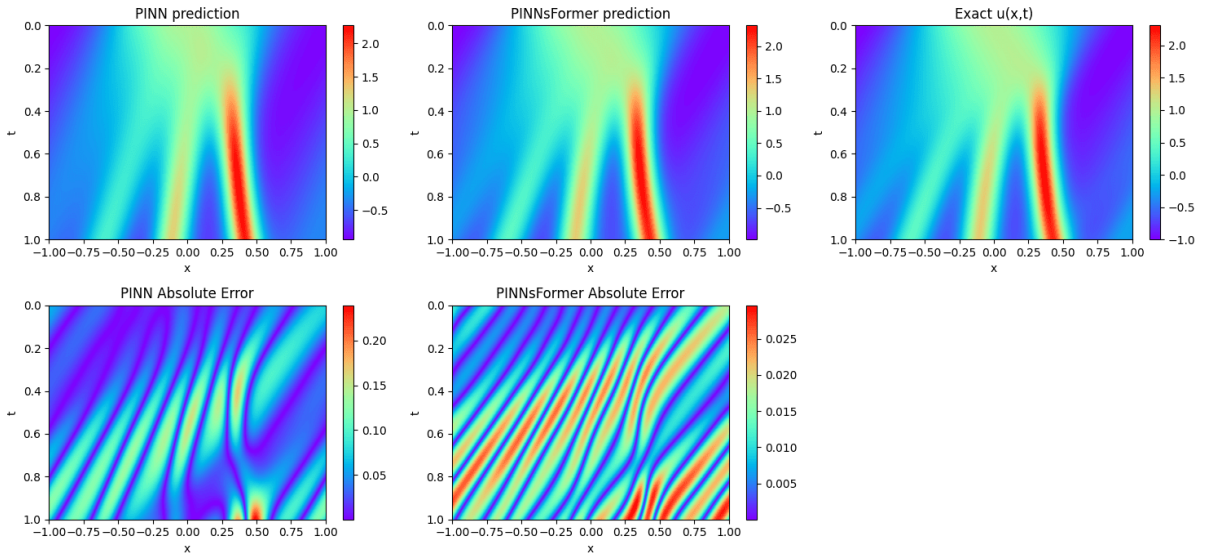


Figure 12: Comparison of the PINN and PINNsFormer predictions with the reference solution for the KdV equation.

The rMAE decreased from 0.0820 with the standard PINN to 0.0184 with PINNsFormer, an improvement of about 78%. A notable aspect of this experiment is that the third-order derivative (u_{xxx}) was computed successfully without any change to the architecture or to the formulation of the method; that is, for this equation at least, the framework generalized to derivatives of higher order than those tested in the original paper.

5 Comparative Analysis and Discussion

Table 12 summarizes the quantitative results for all five equations, and Figure 13 compares the rMAE values of the two models graphically. Because of the large differences among the rMAE values, the

vertical axis of the plot is shown on a logarithmic scale.

Table 12: Comprehensive comparison of results on the five equations.

Equation	PINN		PINNsFormer		rMAE reduction	Better model
	rMAE	rRMSE	rMAE	rRMSE		
Burgers	0.0262	0.0695	0.0120	0.0556	54%	PINNsFormer
Allen–Cahn	0.9916	0.9910	0.3790	0.6100	62%	PINNsFormer
Schrödinger	0.0601	0.0926	0.0032	0.0047	95%	PINNsFormer
Kuramoto–Sivashinsky	0.3012	0.5124	0.0001	0.0001	> 99%	PINNsFormer
KdV	0.0820	0.0857	0.0184	0.0181	78%	PINNsFormer

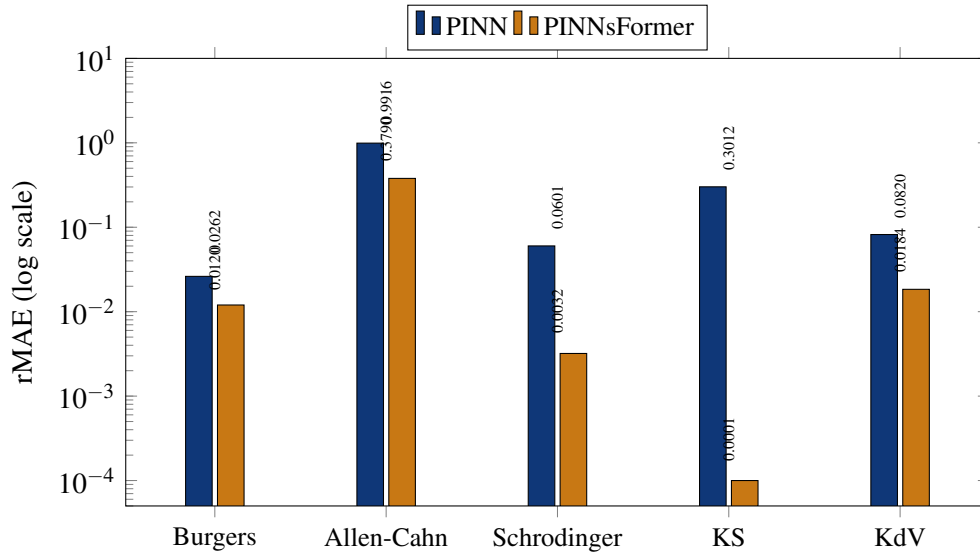


Figure 13: Comparison of the rMAE of the PINN and PINNsFormer models on the five equations (logarithmic scale). On all five equations, the orange bar (PINNsFormer) is shorter than the blue bar (PINN); for Kuramoto–Sivashinsky the difference exceeds three orders of magnitude.

5.1 Overall Observed Patterns

Taking the five equations together, the rMAE improvement ranges from 54% (Burgers) to more than 99% (Kuramoto–Sivashinsky), a fairly wide range that is itself of interest. For the Schrödinger equation, where the standard PINN already performed reasonably well ($\text{rMAE} \approx 0.06$), PINNsFormer reduced the error to 0.0032. For the Allen–Cahn equation, where the standard PINN had effectively failed ($\text{rMAE} \approx 0.99$), PINNsFormer brought this value down to 0.38, a large relative improvement but still a comparatively high error relative to the other equations. Taken together, these two cases suggest that the advantage of the attention-based architecture does not take the same form everywhere: where the PINN fails completely (Allen–Cahn), PINNsFormer can improve the situation without necessarily reaching high accuracy, whereas where the PINN is relatively successful from the outset (Schrödinger), the further improvement occurs at the level of fine-grained accuracy.

The Kuramoto–Sivashinsky equation departs from both patterns: the error of the standard PINN was moderate (0.30), yet PINNsFormer lowered it to 0.0001, the largest improvement of the whole study. Section 4.4.1 examined this case in terms of the match between the physical scale of the problem and the fixed hyperparameters of the sequence generator.

5.2 Computational Cost Analysis

Table 13 reports the training time of each model.

Table 13: Comparison of training times.

Equation	PINN time	PINNsFormer time	Ratio
Burgers	4 min 1 s	28 min 25 s	$\sim 7.1\times$
Allen–Cahn	1 min 28 s	32 min 53 s	$\sim 22.4\times$
Schrödinger	7 min 12 s	49 min 59 s	$\sim 6.9\times$
Kuramoto–Sivashinsky	2 min 53 s	53 min 57 s	$\sim 18.7\times$
KdV	10 min 38 s	1 h 49 min	$\sim 10.3\times$
Mean			$\sim 13.1\times$

6 Conclusion

In this work, PINNsFormer was implemented for five equations (Burgers, Allen–Cahn, nonlinear Schrödinger, Kuramoto–Sivashinsky, and Korteweg–de Vries) and compared with a standard PINN. Four main observations can be summarized as follows. First, on all five experiments the rMAE of PINNsFormer was lower than that of the baseline. Second, the size of the reduction was not uniform, ranging from 54% (Burgers) to more than 99% (Kuramoto–Sivashinsky). Third, the Kuramoto–Sivashinsky equation, the only fourth-order equation in this set, showed the largest improvement, whereas for Allen–Cahn the final error remained high despite a considerable improvement. Fourth, this gain in accuracy was not free: the training time of PINNsFormer was on average about 13 times that of the standard PINN. An inspection of the loss components also showed that the reduction in final error does not always stem from a reduction in the equation residual; for the Schrödinger equation, a large part of the improvement was related to the initial condition.

Several methodological limitations should nevertheless be kept in mind when interpreting these results: all experiments were run with a single random seed, the number and dimensional diversity of the equations studied are limited, GPU memory limits reduced the collocation density for the Kuramoto–Sivashinsky equation, and only one set of hyperparameters was tested for each equation. Within these limitations, the conclusion that emerges is that a sequence-based structure combined with an attention mechanism was able, at least in these five cases, to improve on the accuracy of the standard PINN at a considerable computational cost, but that this improvement did not occur to the same extent or through the same mechanism in all problems. Future work should address multiple random seeds, an ablation over the sequence length k and time step Δt , and equal-compute-budget comparisons.

References

- [1] Zhao, Z., Ding, X., & Prakash, B. A. (2024). *PINNsFormer: A Transformer-Based Framework for Physics-Informed Neural Networks*. International Conference on Learning Representations (ICLR).
- [2] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*. Journal of Computational Physics, 378, 686–707.
- [3] Lagaris, I. E., Likas, A., & Fotiadis, D. I. (1998). *Artificial neural networks for solving ordinary and partial differential equations*. IEEE Transactions on Neural Networks, 9(5), 987–1000.
- [4] Lu, L., Jin, P., Pang, G., Zhang, Z., & Karniadakis, G. E. (2021). *Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators*. Nature Machine Intelligence, 3(3), 218–229.
- [5] Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., & Anandkumar, A. (2020). *Fourier neural operator for parametric partial differential equations*. arXiv preprint arXiv:2010.08895.
- [6] Lu, L., Meng, X., Mao, Z., & Karniadakis, G. E. (2021). *DeepXDE: A deep learning library for solving differential equations*. SIAM Review, 63(1), 208–228.
- [7] Rahaman, N., Baratin, A., Arpit, D., Draxler, F., Lin, M., Hamprecht, F., Bengio, Y., & Courville, A. (2019). *On the spectral bias of neural networks*. International Conference on Machine Learning (ICML), 5301–5310.
- [8] Krishnapriyan, A., Gholami, A., Zhe, S., Kirby, R., & Mahoney, M. W. (2021). *Characterizing possible failure modes in physics-informed neural networks*. Advances in Neural Information Processing Systems, 34.
- [9] Wang, S., Teng, Y., & Perdikaris, P. (2021). *Understanding and mitigating gradient flow pathologies in physics-informed neural networks*. SIAM Journal on Scientific Computing, 43(5), A3055–A3081.
- [10] Wang, S., Yu, X., & Perdikaris, P. (2022). *When and why PINNs fail to train: A neural tangent kernel perspective*. Journal of Computational Physics, 449, 110768.
- [11] McClenny, L., & Braga-Neto, U. (2020). *Self-adaptive physics-informed neural networks using a soft attention mechanism*. arXiv preprint arXiv:2009.04544.
- [12] Wang, S., Sankaran, S., & Perdikaris, P. (2022). *Respecting causality is all you need for training physics-informed neural networks*. arXiv preprint arXiv:2203.07404.
- [13] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention is all you need*. Advances in Neural Information Processing Systems, 30.
- [14] Dağ, İ., Irk, D., & Saka, B. (2005). *A numerical solution of the Burgers' equation using cubic B-splines*. Applied Mathematics and Computation, 163(1), 199–211.
- [15] Khalid, N., Abbas, M., Iqbal, M. K., et al. (2020). *A numerical investigation of Caputo time fractional Allen–Cahn equation using redefined cubic B-spline functions*. Advances in Difference Equations, 2020, 158.

- [16] Gardner, L. R. T., Gardner, G. A., Zaki, S. I., & El Sahrawi, Z. (1993). *B-spline finite element studies of the non-linear Schrödinger equation*. Computer Methods in Applied Mechanics and Engineering, 108(3–4), 303–318.
- [17] Lakestani, M., & Dehghan, M. (2012). *Numerical solutions of the generalized Kuramoto–Sivashinsky equation using B-spline functions*. Applied Mathematical Modelling, 36(2), 605–617.
- [18] Jain, P. C., Shankar, R., & Bhardwaj, D. (1997). *Numerical solution of the Korteweg-de Vries (KdV) equation*. Chaos, Solitons & Fractals, 8(6), 943–951.